НЕГОСУДАРСТВЕННОЕ ОБРАЗОВАТЕЛЬНОЕ ЧАСТНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

«МОСКОВСКИЙ ФИНАНСОВО-ПРОМЫШЛЕННЫЙ УНИВЕРСИТЕТ «СИНЕРГИЯ»

Кафедра Инновационных и цифровых технологий

ЛАБОРАТОРНАЯ РАБОТА № 1

Обучающийся Волков Илья Викторович

(ФИО) (подпись)

Преподаватель Сибирев Иван Валерьевич

(ФИО) (подпись)

Москва 2026 г.

СОДЕРЖАНИЕ

Введение 3

1. Сущность CI/CD 3

2. Непрерывная интеграция (CI) 4

3. Непрерывная доставка и развёртывание (CD) 5

4. Платформа GitLab CI/CD 5

5. Архитектура GitLab CI/CD 6

6. Пример пайплайна на GitLab CI 7

7. Сравнение с GitHub Actions и Jenkins 8

8. Argo CD как решение для развёртывания 9

Заключение 10

Список использованных источников 10

ВВЕДЕНИЕ

В условиях стремительного развития информационных технологий современная индустрия разработки программного обеспечения предъявляет всё более высокие требования к скорости выпуска продуктов, их качеству и надёжности. Классические подходы к разработке, основанные на длительных релизных циклах и строгом разделении труда между разработчиками и администраторами, перестали отвечать требованиям бизнеса. На смену им пришла методология DevOps, объединяющая процессы разработки (Development) и эксплуатации (Operations) в единый непрерывный поток создания ценности.

Ключевым элементом практической реализации DevOps являются технологии CI/CD — непрерывной интеграции и непрерывной доставки, которые позволяют автоматизировать весь цикл создания программного продукта: от написания кода до его развёртывания в промышленной среде. Сегодня существует множество инструментов, поддерживающих эти процессы, среди которых одно из лидирующих мест занимает платформа GitLab.

Цель данной работы — рассмотреть сущность технологий CI/CD, изучить их ключевые принципы и практическую реализацию на примере платформы GitLab, а также провести сравнительный анализ с другими популярными решениями: GitHub Actions, Jenkins и Argo CD.

1. СУЩНОСТЬ CI/CD

CI/CD (Continuous Integration / Continuous Delivery / Continuous Deployment) — это совокупность практик и инструментов, обеспечивающих автоматизированный процесс интеграции, тестирования и доставки программного кода. Данная методология является одним из столпов философии DevOps и направлена на ускорение релизного цикла при одновременном повышении качества программного продукта.

Аббревиатура CI/CD объединяет три взаимосвязанных, но различных по смыслу понятия:

- Continuous Integration (CI) — непрерывная интеграция, при которой изменения, вносимые разработчиками, автоматически объединяются в общей ветке кода и проходят проверку на корректность;
- Continuous Delivery (CD) — непрерывная доставка, обеспечивающая готовность кода к развёртыванию в любой момент времени;
- Continuous Deployment (CD) — непрерывное развёртывание, при котором каждое одобренное изменение автоматически попадает в продуктивную среду без участия человека.

Основная идея CI/CD заключается в том, чтобы разработчик, внёсший любое изменение в код, получал обратную связь о его корректности в кратчайшие сроки. Это достигается путём построения автоматизированного конвейера (pipeline), который запускается при каждом изменении исходного кода и последовательно выполняет этапы: сборку, тестирование, анализ качества, упаковку артефактов и развёртывание.

Классический цикл DevOps часто изображается в виде бесконечной петли, включающей этапы Plan (планирование), Code (написание кода), Build (сборка), Test (тестирование), Release (релиз), Deploy (развёртывание), Operate (эксплуатация) и Monitor (мониторинг). Первая половина петли соответствует CI, вторая — CD. Именно CI/CD обеспечивает непрерывное движение по этой петле, превращая разработку из серии дискретных этапов в бесшовный процесс.

2. НЕПРЕРЫВНАЯ ИНТЕГРАЦИЯ (CI)

Непрерывная интеграция — это практика, при которой разработчики регулярно (как правило, несколько раз в день) объединяют свои изменения с основной веткой репозитория. Каждое такое слияние автоматически запускает

процесс сборки и тестирования, что позволяет выявлять ошибки на самых ранних стадиях их появления.

Ключевые принципы CI включают:

- Единый репозиторий исходного кода (обычно на базе Git);
- Автоматизированную сборку проекта при каждом коммите;
- Обязательное прохождение автоматических тестов (модульных, интеграционных, статического анализа кода);
- Быструю обратную связь разработчику о результатах проверки;
- Поддержание основной ветки в работоспособном состоянии.

Среди инструментов, используемых для реализации CI, широкое распространение получили системы сборки (Gradle, Maven, Bazel, Webpack), фреймворки для тестирования (JUnit, Jest, Playwright), а также специализированные CI-серверы — Jenkins, Buildkite, GitHub Actions и GitLab CI/CD.

3. НЕПРЕРЫВНАЯ ДОСТАВКА И РАЗВЁРТЫВАНИЕ (CD)

Если CI отвечает за качество кода на этапе разработки, то Continuous Delivery и Continuous Deployment отвечают за его путь к конечному пользователю. Различие между этими понятиями заключается в степени автоматизации финального шага.

Непрерывная доставка (Continuous Delivery) предполагает, что после прохождения всех этапов конвейера артефакт приложения готов к развёртыванию в любой момент, однако решение о публикации в продакшен принимает человек. Это особенно важно для систем с высокими требованиями к стабильности.

Непрерывное развёртывание (Continuous Deployment) идёт ещё дальше — каждое одобренное изменение автоматически попадает в продуктивную среду, если оно прошло все автоматические проверки. Такой

подход возможен только при наличии комплексной системы автоматизированного тестирования и мониторинга.

Для реализации CD широко применяются контейнерные технологии (Docker), системы оркестрации контейнеров (Kubernetes), инструменты управления инфраструктурой как кодом (Terraform), платформы бессерверных вычислений (AWS Lambda), а также инструменты GitOps, например Argo CD. Мониторинг приложений в продакшене обеспечивают Prometheus и Datadog.

4. ПЛАТФОРМА GITLAB CI/CD

GitLab — это веб-платформа для управления репозиториями Git и всем жизненным циклом разработки программного обеспечения. В отличие от многих конкурентов, GitLab позиционируется как *единая DevOps-платформа*, объединяющая в одном продукте функции управления кодом, планирования задач, CI/CD, управления безопасностью, контейнерным реестром и мониторингом.

Платформа была создана в 2011 году Дмитрием Запорожцем и Валерием Сизовым. Сегодня GitLab распространяется в нескольких редакциях: Community Edition (CE) с открытым исходным кодом, Enterprise Edition (EE) с расширенным функционалом, а также в облачном варианте GitLab.com. Платформу можно развернуть как на собственных серверах (self-hosted), так и использовать облачный вариант — это особенно важно для организаций с повышенными требованиями к безопасности данных.

GitLab CI/CD — встроенный в GitLab компонент для автоматизации процессов сборки, тестирования и развёртывания. Он позволяет описывать весь конвейер разработки в виде декларативного YAML-файла, хранящегося непосредственно в репозитории проекта. Такой подход полностью соответствует принципу «инфраструктура как код».

5. АРХИТЕКТУРА GITLAB CI/CD

Архитектура GitLab CI/CD состоит из нескольких ключевых компонентов:

GitLab Server — центральный компонент, управляющий репозиториями, пользователями, проектами и координирующий выполнение конвейеров. На нём хранятся исходные коды, определения пайплайнов и логи их выполнения.

GitLab Runner — исполнительный компонент, который непосредственно выполняет задания конвейера. Runner может быть установлен на любой машине и способен запускать задания в различных окружениях: в shell, в Docker-контейнерах, в виртуальных машинах или в кластерах Kubernetes.

Файл .gitlab-ci.yml — конфигурационный файл, размещаемый в корне репозитория. В нём описываются все этапы (stages) конвейера и задания (jobs), которые должны выполняться на каждом этапе.

Принцип работы GitLab CI/CD основан на событийной модели. При каждом push в репозиторий, создании merge request, запуске по расписанию или ручном триггере GitLab Server анализирует файл .gitlab-ci.yml и формирует конвейер — упорядоченный набор заданий. Задания передаются свободным Runner-ам, которые выполняют их в изолированных окружениях и возвращают результаты (логи, артефакты, статусы) обратно на сервер.

Важной особенностью GitLab CI/CD является поддержка артефактов и кеширования. Артефакты — это файлы, созданные в ходе выполнения задания (скомпилированные бинарники, отчёты тестирования), которые могут передаваться между этапами и сохраняться для последующего анализа.

6. ПРИМЕР ПАЙПЛАЙНА НА GITLAB CI

Рассмотрим практический пример конфигурации .gitlab-ci.yml для веб-приложения на Node.js с этапами сборки, тестирования, сканирования безопасности и развёртывания в Kubernetes-кластер:

```
# Базовый Docker-образ для всех заданий
image: node:20-alpine

# Определение этапов конвейера
stages:
  - build
  - test
  - security
  - deploy

# Глобальные переменные
variables:
  DOCKER_IMAGE: $CI_REGISTRY_IMAGE:$CI_COMMIT_SHA

# Кеширование зависимостей npm
cache:
  paths:
    - node_modules/

# Этап сборки: компиляция проекта
build-job:
  stage: build
  script:
    - npm ci
    - npm run build
  artifacts:
    paths:
```

```
      - dist/
    expire_in: 1 week

# Модульные тесты
unit-tests:
  stage: test
  script:
    - npm run test:unit
  coverage: '/Coverage: \d+\.\d+%/'

# E2E тесты с Playwright
e2e-tests:
  stage: test
  script:
    - npx playwright test

# Сканирование уязвимостей (SAST)
sast-scan:
  stage: security
  script:
    - npm audit --audit-level=high
  allow_failure: true

# Развёртывание в production
deploy-prod:
  stage: deploy
  image: bitnami/kubectl:latest
  script:
    - kubectl set image deployment/app app=$DOCKER_IMAGE
    - kubectl rollout status deployment/app
  environment:
    name: production
```

```
    url: https://app.example.com
  only:
    - main
  when: manual
```

В приведённом примере демонстрируются ключевые возможности GitLab CI/CD. Конвейер разделён на четыре этапа, задания внутри одного этапа выполняются параллельно, а между этапами — последовательно. Этап test одновременно запускает модульные тесты и E2E-тесты, что существенно сокращает общее время выполнения. Артефакты сборки передаются между заданиями автоматически. Развёртывание в production происходит только при коммите в ветку main и требует ручного подтверждения (when: manual) — это реализация именно Continuous Delivery, а не полностью автоматического развёртывания. Параметр environment связывает задание с логической средой, что позволяет GitLab отслеживать историю развёртываний и быстро выполнять откаты при необходимости.

7. СРАВНЕНИЕ С GITHUB ACTIONS И JENKINS

Помимо GitLab CI/CD, на рынке существуют и другие популярные инструменты автоматизации, каждый со своими сильными и слабыми сторонами. Рассмотрим особенности наиболее распространённых альтернатив.

GitHub Actions — система CI/CD, интегрированная в GitHub. Конфигурация описывается в YAML-файлах, размещённых в каталоге .github/workflows. Сильной стороной является огромный каталог готовых компонентов (Actions Marketplace), содержащий десятки тысяч переиспользуемых действий. GitHub Actions оптимален для open-source проектов и команд, уже использующих GitHub, однако для self-hosted сценариев требует подписки на GitHub Enterprise.

Jenkins — один из старейших и наиболее известных инструментов CI/CD, выпущенный в 2011 году. Отличается исключительной гибкостью благодаря экосистеме из более чем 1800 плагинов, покрывающих практически любые сценарии автоматизации. Конфигурация может осуществляться как через графический интерфейс, так и декларативно через Jenkinsfile на языке Groovy. Jenkins устанавливается на собственные серверы и требует значительных усилий по настройке и сопровождению, но взамен предоставляет практически неограниченные возможности кастомизации.

Основные отличия этих систем удобно представить в виде сравнительной таблицы:

Критерий	GitLab CI/CD	GitHub Actions	Jenkins
Тип решения	Единая платформа	Встроен в GitHub	Отдельный сервер
Конфигурация	.gitlab-ci.yml	.github/workflows/*.yml	Jenkinsfile (Groovy)
Self-hosted	Да, полноценно	Ограниченно (Enterprise)	Да, основной режим
Исполнители	GitLab Runner	GitHub Runner	Agent / Node
Плагины	Встроенные модули	Marketplace Actions	1800+ плагинов
DevSecOps	Встроенный (SAST, DAST)	Через Marketplace	Через плагины
Порог входа	Средний	Низкий	Высокий

Выбор конкретного инструмента зависит от специфики проекта. GitLab CI/CD оптимален для команд, которым нужна единая DevOps-платформа «из коробки» с поддержкой self-hosted развёртывания и встроенными средствами безопасности. GitHub Actions подходит проектам, уже размещённым на GitHub, особенно open-source. Jenkins остаётся лучшим выбором для сложных

инфраструктур с нестандартными требованиями, где важна максимальная гибкость настройки.

8. ARGO CD КАК РЕШЕНИЕ ДЛЯ РАЗВЁРТЫВАНИЯ

Argo CD — это декларативный инструмент непрерывного развёртывания для Kubernetes, реализующий парадигму *GitOps*. Ключевая идея GitOps заключается в том, что желаемое состояние инфраструктуры и приложений описывается в Git-репозитории, а специальный агент автоматически приводит реальное состояние кластера в соответствие с описанным. Проект Argo CD был создан компанией Intuit в 2018 году и в настоящее время является выпускным проектом фонда Cloud Native Computing Foundation (CNCF).

Принципиальное отличие Argo CD от классических CI/CD-систем заключается в модели работы. Если GitLab CI или Jenkins используют push-модель (сервер CI/CD подключается к Kubernetes и отправляет в него изменения), то Argo CD использует pull-модель: агент, работающий внутри кластера, периодически опрашивает Git-репозиторий и самостоятельно применяет обнаруженные изменения. Такая архитектура обеспечивает ряд важных преимуществ:

- Повышенная безопасность — учётные данные Kubernetes не выходят за пределы кластера;
- Автоматическое обнаружение отклонений (drift detection) — если кто-то изменил ресурсы в кластере вручную, Argo CD вернёт их к описанному в Git состоянию;
- Полная прозрачность — вся история изменений хранится в Git и доступна для аудита;
- Простота отката — достаточно откатить коммит в Git, и изменения автоматически применятся к кластеру.

На практике Argo CD часто используется в связке с GitLab CI/CD: GitLab отвечает за этапы сборки, тестирования и публикации Docker-образов в реестр, после чего обновляет в отдельном Git-репозитории манифесты Kubernetes, указывая новую версию образа. Argo CD обнаруживает эти изменения и применяет их к кластеру. Такое разделение ответственности между CI-системой и CD-инструментом считается современной best-practice в Kubernetes-ориентированной разработке.

Argo CD предоставляет удобный веб-интерфейс для визуализации состояния приложений, CLI для автоматизации и поддерживает различные шаблонизаторы манифестов (Helm, Kustomize, Jsonnet). Инструмент также включает поддержку стратегий прогрессивного развёртывания (canary, blue-green) через расширение Argo Rollouts.

ЗАКЛЮЧЕНИЕ

В ходе выполнения настоящей работы были рассмотрены ключевые аспекты технологий CI/CD как неотъемлемой составляющей методологии DevOps. Установлено, что непрерывная интеграция, непрерывная доставка и непрерывное развёртывание образуют единый автоматизированный конвейер, позволяющий ускорить процесс разработки и повысить качество выпускаемого программного обеспечения.

Особое внимание было уделено платформе GitLab как комплексному инструменту, реализующему принципы CI/CD в рамках единой DevOps-экосистемы. На практическом примере .gitlab-ci.yml показаны возможности построения многоэтапного конвейера с параллельными заданиями, кешированием, артефактами и ручным развёртыванием.

Проведённое сравнение GitLab CI/CD с GitHub Actions и Jenkins показало, что каждое решение обладает собственной нишей применения: GitLab оптимален для единой платформы, GitHub Actions — для open-source проектов, Jenkins — для сложных и нестандартных инфраструктур. Отдельно

рассмотрен инструмент Argo CD, реализующий современный подход GitOps и прекрасно дополняющий классические CI-системы в Kubernetes-окружениях.

Таким образом, технологии CI/CD в целом и платформа GitLab в частности представляют собой важнейший инструмент современной разработки программного обеспечения, без которого невозможно представить эффективную работу профессиональной команды в условиях высокой конкуренции и быстро меняющихся требований рынка.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Ким Дж., Дебуа П., Уиллис Дж., Хамбл Дж. Руководство по DevOps. — М.: Манн, Иванов и Фербер, 2018. — 416 с.
2. Хамбл Дж., Фарли Д. Непрерывное развёртывание ПО: автоматизация процессов сборки, тестирования и внедрения новых версий программ. — М.: Вильямс, 2011. — 432 с.
3. Форсгрен Н., Хамбл Дж., Ким Дж. Ускоряйся! Наука DevOps. — М.: ИД «Питер», 2019. — 288 с.
4. Официальная документация GitLab [Электронный ресурс]. — URL: https://docs.gitlab.com (дата обращения: 24.04.2026).
5. Документация по GitLab CI/CD [Электронный ресурс]. — URL: https://docs.gitlab.com/ee/ci/ (дата обращения: 24.04.2026).
6. Документация GitHub Actions [Электронный ресурс]. — URL: https://docs.github.com/actions (дата обращения: 24.04.2026).
7. Документация Jenkins [Электронный ресурс]. — URL: https://www.jenkins.io/doc/ (дата обращения: 24.04.2026).
8. Документация Argo CD [Электронный ресурс]. — URL: https://argo-cd.readthedocs.io (дата обращения: 24.04.2026).
9. Чакон С., Штрауб Б. Pro Git. — 2-е изд. — Apress, 2014. — 458 с.